

プログラミング演習スライド

金子知適

東京大学総合文化研究科 (kaneko@acm.org)

week4

実体と参照

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

変数とは

「値」を保持する。名前を持つ。値の**読み出し**と**書き込み**ができる。
計算手順の表記に利用 i.e., コンピュータの能力の仮定の一環



比喻: 変数 ~ メモ帳

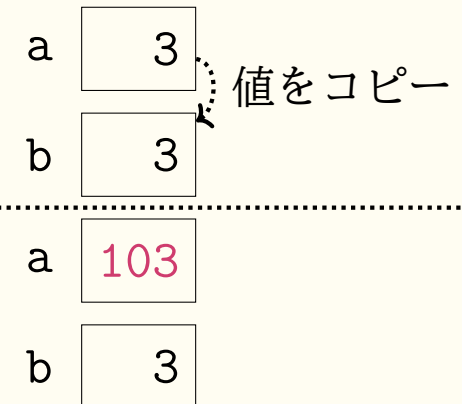
- 読む: 一番上の紙を読む
- 書く: 紙を一枚捨てて、次の用紙に値を記入

数

実体 (方便) を変数に記録
変数の複製 直観通り

a = 3
b = $\underbrace{a}_{3}$

a = $\underbrace{a + 100}_{103}$



結果

- a → 103
- b → 3

実体と参照

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

変数とは

「値」を保持する。名前を持つ。値の**読み出し**と**書き込み**ができる。
計算手順の表記に利用 i.e., コンピュータの能力の仮定の一環



比喩: 変数 ~ メモ帳

- 読む: 一番上の紙を読む
- 書く: 紙を一枚捨てて、次の用紙に値を記入

配列

「**参照**」 (new!) を変数に記録
変数の複製 参照の複製 (!)

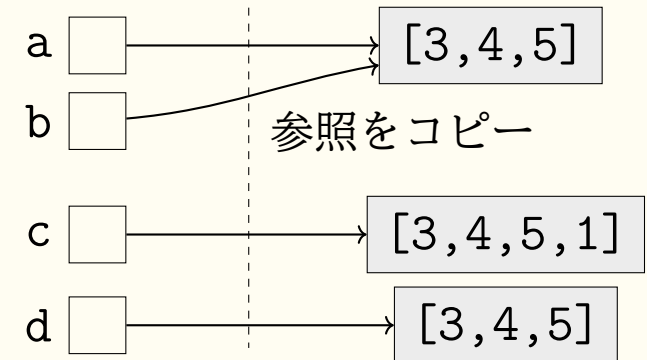
比喩

- 実体 — 銀行口座のお金
- 参照 — 通帳

```
a = [3,4,5]
b = a # 要注意操作
c = a + [1]
d = [3,4,5]
```

結果

- a → [3,4,5]
- b → [3,4,5]
- a==b==d → True



参照をコピー

ヒープ領域

変数 a, b は実体を共有

要区別: 「通帳」のコピー (b) と, 「口座」の新規開設 (c, d)

配列の破壊的変更

プログラミング演習
スライド

金子

配列を返す関数

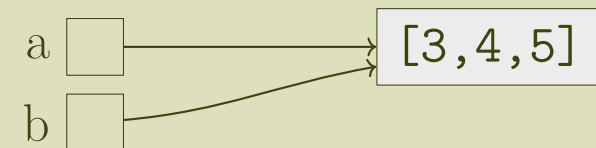
2次元配列

配列 seq の破壊的変更手段の例 — 実体に作用 —

- `seq[i] = 式` 要素`seq[i]`を式の値に書き換え
- `seq.append(式)` 末尾に式の値を持つ要素を追加
- `seq.sort()` 要素を昇順に並び替え

変更前の状況

```
a = [3,4,5]
b = a
```

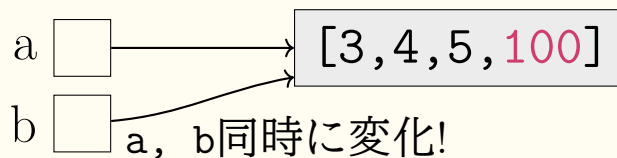


append

```
a.append(100)
```

結果

- `a` → `[3, 4, 5, 100]`
- `b` → `[3, 4, 5, 100]`

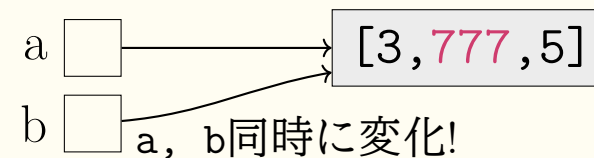


要素書き換え

```
b[1] = 777
```

結果

- `a` → `[3, 777, 5]`
- `b` → `[3, 777, 5]`



要注意: 関数の引数も参照を複製 = 実体を共有

bug の要因なので, 破壊的変更は慎重に.

配列を作成する関数

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

配列を作成する関数の基本パターン

```
def 関数名(引数1,...):  
    配列 = ita.array.make1d(長さ) # または ita.array.make1d(長さ, 初期値)  
    for i in range(len(配列)):  
        配列[i] = 式      # 各要素の値を設定  
    return 配列
```

例1

長さ n で要素がすべて 1 の配列を返す関数 `ones(n)` を、基本パターンで作成せよ

```
def ones(n):  
    seq = ita.array.make1d(n)  
    for i in range(n):  
        seq[i] = 1  
    return seq
```

例2

0 から $n-1$ までの整数を1ずつ要素として持つ配列を返す関数 `iota_alt(n)` を作成せよ

```
def iota_alt(n):  
    seq = ita.array.make1d(n)  
    for i in range(n):  
        seq[i] = i  
    return seq
```

.....ここまででいったん演習

2次元配列とリストのリスト

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

基本パターン

エラステネスの篩

配列とリストの一般的な使い分け

2次元配列[†]

- 形状 長方形
- 大きさ 基本固定
- 要素の型 同一

リストのリスト

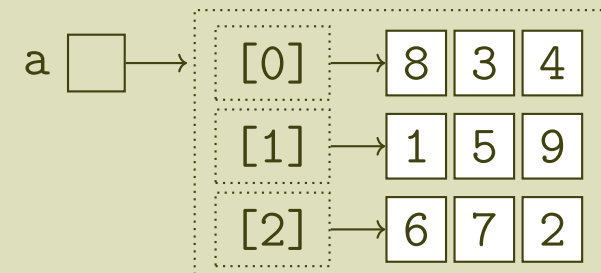
- 形状 自由
- 大きさ 基本固定
- 要素の型 いろいろ

[†] 実用の Python では `numpy.ndarray` が標準的

2次元配列としての「リストのリスト」

```
a = [[8,3,4],  
      [1,5,9],  
      [6,7,2]]
```

```
>>> a[0]          1  
[8,3,4]          # list 2  
>>> a[0][0]       3  
8               # int 4
```



使用例 行列, xy 座標の高さ, ゲームボード

2次元配列 全要素を読む

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

基本パターン

エラトステネスの篩

復習 ◀

1次元配列の基本パターン

```
def 関数名(seq, 引数2, ...):  
    変数 = 式 # 初期値  
    for i in range(len(seq)):  
        変数 = seq[i] を使う式  
    return 変数
```

例 全要素の和

```
def sum_of(seq):  
    total = 0  
    for i in range(len(seq)):  
        total = total + seq[i]  
        # ループ不変条件 total は seq[i] までの和  
    return total
```

2次元配列の基本パターン

```
def 関数名(seq2d, 引数2, ...):  
    変数 = 式 # 初期値  
    for r in range(len(seq2d)): # 行r  
        for c in range(len(seq2d[0])): # 列c  
            変数 = seq2d[r][c] を使う式  
                        読む  
    return 変数
```

2重ループ ◀ になったが、本質は1次元と共通

例 要素の最小値

```
def min_in_mat(a):  
    minimum = a[0][0]  
    for r in range(len(a)):  
        for c in range(len(a[0])):  
            if minimum > a[r][c]:  
                minimum = a[r][c]  
            # 不変条件 a[r'][c'] 内の最小  
            #  $r' < r$  or  $(r' = r, c' \leq c)$   
    return minimum
```

エラトステネスの篩

教養

プログラミング演習
スライド

金子

配列を返す関数

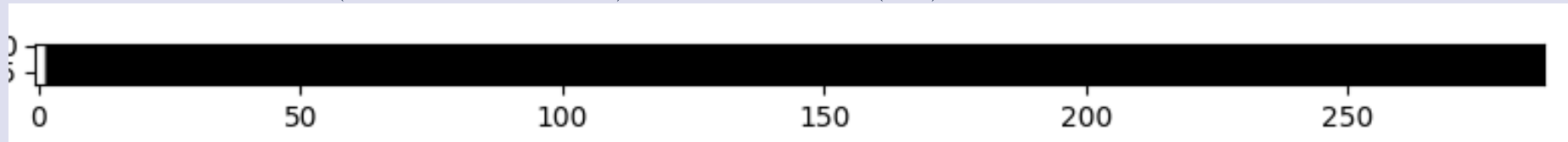
2次元配列

基本パターン

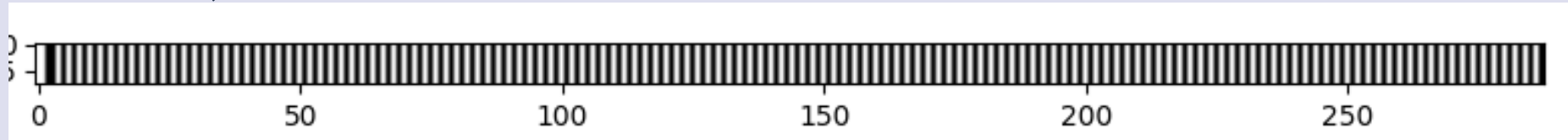
エラトステネスの篩

アルゴリズム概略

- 0, 1 を白にmark (素数ではない). 他は不明 (黒)



- 2を残して, 2の倍数をmark

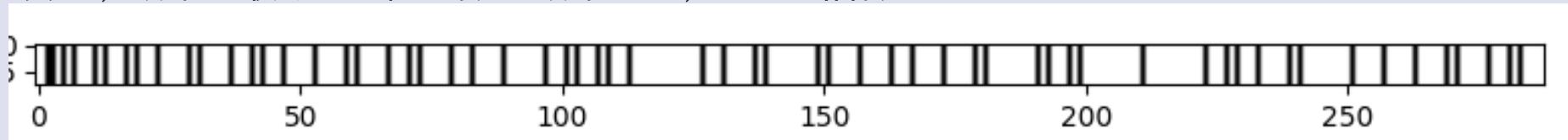


- 同様に, 3を残して, 3の倍数をmark



- 4は, mark済みなので, なにもせず5に進む

- 順に, 残る最小の紫の数を残して, その倍数をmark



- 残った黒が素数

効率 — 大きな素数はまばらなので, mark する頻度も減ってゆく

エラトステネスの篩 — 実装

プログラミング演習
スライド

金子

配列を返す関数

2次元配列

基本パターン

エラトステネスの篩

篩の1ステップ相当 `mark_multiples(seq, n)`

真偽値の配列 `seq` について `n` より大きな `n` の倍数 `i` について `seq[i]` を `True` とせよ. それ以外の要素は変更しない.

実装例1 1つずつ確認

```
def mark_multiples(seq, n):  
    for i in range(len(seq)):  
        if i % n == 0 and i > n:.... †  
            seq[i] = True ..... †
```

実装例2 `n` とばしに処理

```
def mark_multiples(seq, n):  
    i = 2*n  
    while i < len(seq):  
        seq[i] = True ..... †  
        i += n
```

どちらの実装が効率的?

→ `n` が大きいと, 実装例2の方が効率的

`N=len(seq)` として

- † の行 N/n 回程度 (両方)
- ‡ の行 N 回程度 (実装例1のみ存在)